Circuits Updates - March 2024

We report a number of developing ideas on the Anthropic interpretability team, which might be of interest to researchers working actively in this space. Some of these are emerging strands of research where we expect to publish more on in the coming months. Others are minor points we wish to share, since we're unlikely to ever paper about them.

We'd ask you to treat these results like those of a colleague sharing some thoughts or preliminary experiment few minutes at a lab meeting, rather than a mature paper.

New Posts

- Using Features For Fast Circuit Identification
- Update on Dictionary Learning Improvements

Updates

- Research By Other Groups

Using Features For Easy Circuit Identification

Joshua Batson, Brian Chen, Andy Jones

Finding circuits in language models can be a labor-intensive process. You specify a behavior of interest, such a indirect-object identification, inequality identification, or factual association, then assemble a dataset with hun or thousands of examples of that behavior. To find model components which are important for a prediction, y ablate or patch them in turn on each example. This produces a list of components, and further investigations c reveal their interactions on the assembled dataset.

Features produced by sparse autoencoders (SAEs) offer an alternative to behavior-specific dataset curation. Ke aspects of the full data distribution, as processed by the model, are encapsulated in the dictionary. This allows generate rich hypotheses from a single example text, and to study the more general function of the revealed components in terms of other features they interact with.

We illustrate this approach by investigating the attribute-extraction process following *Nanda et al.* (see also *G al.*, *Meng et al.*). The authors studied completions of the sentence "Fact: [ATHLETE NAME] plays the sport of" fo collection of ~1500 [ATHLETE NAME]s playing basketball, baseball, or tennis. The authors found three distinct

- token_concatenation in which information from all tokens of [ATHLETE NAME] are moved to the final t of the subject
- subject_enrichment (in the language of *Geva et al.*) in which early MLP layers add information to the re stream for the final subject token.
- extract_sport in which attention heads (primarily one specific head) selects the sport information from final subject token and transmits it to the final token. Composing the OV circuit of the specific head wi unembedding matrix even produces a mechanistic probe for sport.

We investigate an 18-layer model, with a sparse autoencoder with expansion factor of ~256 trained on the lay residual stream (as in Cunningham et al.).

Detecting Important Features with Attribution

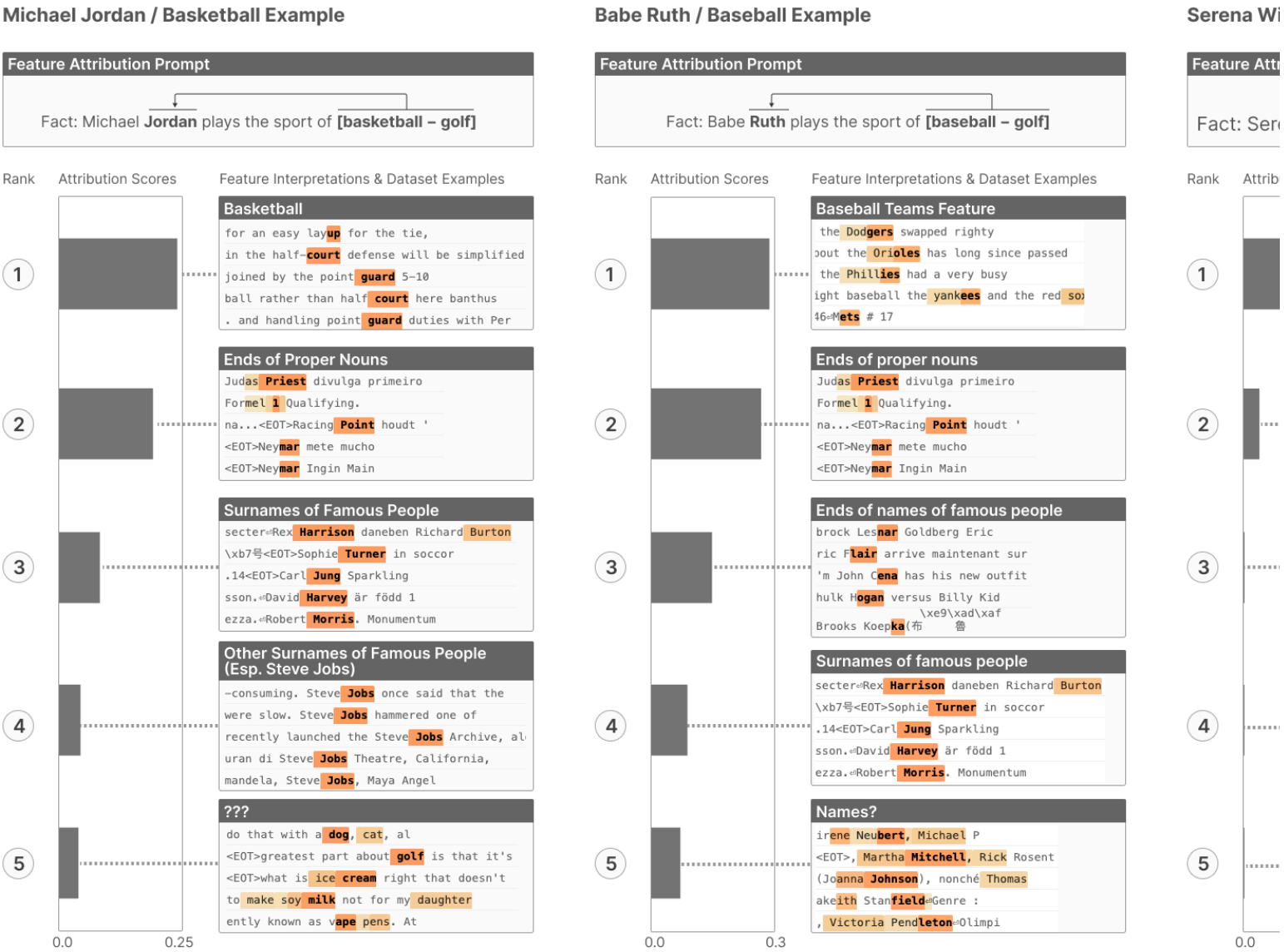
The first step of our investigation was determining which features are driving the behavior in our task. We automatically identify basketball, baseball, and tennis features in the residual stream, learned by our SAE, whic to play a key role.

We begin with the example text "Fact: Michael Jordan plays the sport of", and compute an attribution score fo autoencoder feature in the layer 9 residual stream at the position [Jordan] with respect to the logit difference between completions of [basketball] and [golf]:

$$\mathrm{attr}_i := a_i \mathbf{d_i} \cdot \nabla_x \mathcal{L},$$

where a_i is the activation of feature i, $\mathbf{d}_i$ is the dictionary (decoder) vector for feature i in the SAE, and $\nabla_x \mathcal{L}$ i gradient of the logit difference between [basketball] and [golf] on the prediction of the final token with respe the residual stream on the token [Jordan] in layer 9 where the SAE is defined. (We use the logit difference bet two sports to identify the features that produce basketball specifically, and avoid those which might produce a sports in general.)

There is a clear top feature by attribution. The top-activating dataset examples (from a feature visualization precomputed on the training mix) include basketball-related words such as "layup", "half-court", and "point gu Since each feature's activation is a thresholded affine function on the residual stream $\mathrm{relu}(E_i x + b_i)$ (where that feature's vector in the encoder and b_i is that feature's bias), we have from a single example isolated a bas related probe ($E_i x + b_i > 0$). This also indicates that the subject_enrichment phase has progressed by layer 9 that information about the sport is present in the token.



We repeat this analysis for "Babe Ruth" for baseball and "Serena Williams" for tennis, finding in each case the t feature by attribution fired on baseball ("Dodgers", "Orioles", "yankees") and tennis ("WTA")-related texts.

We note a number of features that respond to ends of names or surnames ("Judas Priest" in one, "John Cena" another, "Carl Jung" in a third) which have significant attribution for "Jordan" and "Ruth" but not "Williams". W hypothesized that this might be due to the model having learned to associate "tennis" with the first name "Ser but with full names for "Michael Jordan" and "Babe Ruth". We took a handful of athlete names, and asked the for completions of "Fact: [ATHLETE FIRSTNAME] Smith plays the sport of"; we found that the model's predictio matched that for [ATHLETE NAME] in exactly the cases where the "Judas Priest" feature had low attribution on original text, and that the predictions were damaged in the cases where the "Judas Priest" feature had high attribution on the original text. The presence and effects of this feature provides additional evidence that the r "detokenizes" its input in early layers.

Detecting Attention Heads that Act on Features

We then looked for attention heads who produced a large direct logit effect from the basketball feature (i.e., t difference in the [basketball] and [golf] entries of is large). We found many such heads, across many layers, wi top five heads belonging to layers 11, 13, and 14. The top logit effects of each head are basketball-related ("ba "NBA"). Moreover, the same set of heads produces baseball-related words on the baseball feature and tennis- words on the top tennis feature. This parallels the discovery of a special head in Nanda et al. which acted as a for sport, as well as the more recent identification of many "Subject Heads" in Chughtai et al., which found tha heads were involved in extracting attributes from the subject in factual recall, producing the correct output via constructive interference.

Effect of Features as Mediated by Heads

Feature Interpretations & Dataset Examples

Basketball

for an easy layup for the tie,
in the half-court defense will be simplified
joined by the point guard 5-10
ball rather than half court here banthus
. and handling point guard duties with Per

Baseball Teams Feature

the Dodgers swapped righty
out the Orioles has long since passed
the Phillies had a very busy
ight baseball the yankees and the red so
46⇦Mets # 17

Tennis

itova (WTA 10)
del torneo WTA de Doha
zimmermann (WTA-179) heeft
—⇨Serena Williams Birthday : 2
dal supervisore WTA sul posto.

Direct Effect of *Feature* → *Head X* → *Logit* Path

Basketball → L11H1		Basketball → L11H2		Basketball → L14H1		Basketbal	
NBA	30.29	Basketball	32.08	NBA	30.07	Basketbal	
basketball	25.16	NBA	31.92	basketball	25.38	basketbal	
Basketball	24.36	basketball	28.87	Basketball	25.28	NBA	
basket	19.87	basket	23.31	basket	18.68	basket	
Maya	19.81	Lakers	18.87	Lakers	16.67	player	

Baseball → L11H1		Baseball → L11H2		Baseball → L14H1		Baseball-	
baseball	29.75	Baseball	29.15	Baseball	28.55	Baseball	
MLB	28.79	baseball	27.10	baseball	27.01	baseball	
Baseball	27.90	MLB	25.84	MLB	26.93	MLB	
Yankees	24.56	pitcher	23.52	Yankees	23.03	batting	
pitcher	18.91	batting	22.14	pitcher	20.25	softball	

Tennis → L11H1		Tennis → L11H2		Tennis → L14H1		Tennis →	
tennis	26.72	Tennis	33.38	Tennis	26.38	Tennis	
Tennis	26.48	tennis	31.03	tennis	24.94	tennis	
ATP	17.65	tournament	19.79	golf	15.58	Tournam	
wim	14.47	Tournament	19.74	Croatia	15.45	tournam	
Serbian	13.72	tournaments	18.10	Croatian	14.91	tournam	

We hope this exercise demonstrates the utility of using SAEs to kickstart circuit analysis.

Update on Dictionary Learning Improvements

Adly Templeton, Tom Conerly, Jonathan Marcus, and Tom Henighan

In previous updates, we've mentioned a number of interventions which we found to be meaningful improvem SAE optimization at the time. Since then, we have made a number of other changes, including bug fixes. These changes have improved optimization and decreased our training losses. With these changes we no longer see meaningful improvements from the following interventions.

- **Ghost Grads**. We no longer see ghost grads decreasing training loss even on 1L models. The training l roughly equal, which doesn't justify the increase in flops. We have some evidence our implementation ghost grads causes loss spikes in our current training setup. (From January and February updates)
- Setting **Adam Beta parameters** to (0, 0.9999) versus the default values of (0.9, 0.999) leads to similar t loss. (From February update)
- **Pruning features** which have a decoder norm < 0.99. Other changes have led to much fewer features are dead or have decoder norm < 0.99, such that this is no longer a meaningful improvement. (From February update)

While these were improvements to the training process we used at the time, other changes have superseded t We intend to publish these changes in a future update once we've vetted them further.

Research By Other Groups

Joshua Batson, Nicholas L Turner, Brian Chen, and Adam Pearce

Finally, we'd like to highlight a selection of recent work by a number of researchers at other groups which we beli be of interest to you if you find our papers interesting.

OTHELLO AND AUTOMATA

In "Dictionary Learning Improves Patch-Free Circuit Discovery in Mechanistic Interpretability: A Case Study on Othello-GPT", He, Ge, Tang, Sun, Cheng, and Qiu use sparse autoencoders as a tool to investigate circuits in a trained to play legal games of Othello (see Li et al.). They train separate SAEs on word embeddings, MLP outpu attention outputs, and find features corresponding to specific states for cells (current move, current state, lega move in) as one moves through the layers. They identify these features using a visualization toolkit which may useful for other people working on Othello. They then ask how, on specific dataset examples, the features in la layers are computed as functions of features in earlier layers by multiplying through attention OV and MLP we (with a fixed gating mask). For example, an MLP output feature in layer 1 that generally corresponds to cell c-3 "mine" is computed on a specific example where it gets flipped as an AND of three attentional features – d-2 " c-2 "theirs", and b-4 "current" – which are in turn OV outputs of embedding features corresponding to each o move. They also decompose attention scores in terms of features, and find mechanisms supporting heads that exclusively attend to "mine" and "theirs" moves.

As the authors note, the L0-norm of feature activations is quite large (means of ~60–100 for the different SAEs d_model of 128), with near-zero reconstruction error. In our experience, such dictionaries tend to be polysema lower activations. We would also be curious to see the feature interpretations investigated using the activation specificity/sensitivity analysis in Bricken et al. It would also be interesting to see followup work with sparser dictionaries, which might make the circuit analyses crisper and more explanatory with respect to ablations.

Othello may also prove to be a useful laboratory for investigating the more general class of highly parallel algo implemented by transformers. The naive algorithm for finding legal moves of Othello takes N steps of sequent computation to determine the board state after N moves; simply play the game, flipping stones after each mo However, an L-layer transformer only allows for ~2L sequential calculations. For games with N > 2L, well-perfo transformers must have learned a different algorithm. A similar phenomenon was explored in detail last year in Transformers Learn Shortcuts to Automata by Liu, Ash, Goel, Krishnamurthy, and Zhang, who found that "a low Transformer can represent the computations of any finite-state automaton (thus, any bounded-memory algori by hierarchically reparameterizing its recurrent dynamics." They construct transformers with width polynomial number of automaton states T and depth O(log(T)) in general. They find solutions with depth O(1) often exist, show that these shortcut solutions can be learned in practice. In recent related work, Transformers, parallel computation, and logarithmic depth, Sanford, Hsu, and Telgarsky, showed that k-hop induction tasks, which na require k sequential iterations of the induction mechanism, can be implemented by transformers of depth log(would be quite interesting to see a feature-based analysis like He et al. applied to automata, and, conversely, t identify the shortcut strategies being learned by Othello transformers.

COMPUTATION IN SUPERPOSITION

"Towards a Mathematical Framework of Computation in Superposition" describes a set of asymptotically optin constructions of transformer layers that implement many-to-many boolean functions of sparse inputs. In their setting, each input variable is assigned an embedding vector in the residual stream, the full boolean state is represented as a sum of the vectors corresponding to True variables: , and the output of the transformer is transformed back into a boolean vector by multiplication by an unembedding and then rounding to 0/1.

Their MLP constructions provide an explanation for the empirical result that arbitrary XORs of input features ca read off from residual stream representations. They also suggest an interpretation of attention circuits as coun "skip feature bigrams" which motivates one of their constructions. Some notable results are highlighted below engines of these proofs are (1) the fact that random unit vectors in high-dimensions have inner product of orc (2) distortion bounds à la Johnson-Lindenstrauss.

- When boolean features are **not** stored in superposition, a single MLP layer using the U-AND construct produce an output where **all** pairwise ANDs of **all** boolean input variables can be linearly read out wit small error (!)
- When those features **are** stored in superposition, a similar construction can compute a number of AND asymptotically close to the number of weight parameters assuming that the input features are sufficier sparse.
- An MLP layer construction can compute all ANDs with a fan-in of k (also with input feature sparsity).
- An MLP layer construction using a quadratic nonlinearity (instead of ReLU) can compute a number of A that's superpolynomial (!) in the number of MLP units
- When skip feature bigrams are in superposition, a QK construction can compare counts between a nur these asymptotically close to the number of weight parameters. This performs half of the function of tl attention layer. They leave representing skip feature *trigrams* (including the outputs) as future work.

In line with the setting above, the authors say a property is "linearly represented" in the residual stream at a ce point if it is linearly accessible up to an additive error. That is, some boolean function is represented in some if there exists a covector such that on all (k-sparse) inputs. It is possible for two vectors and to share many properties without being close in an or MSE sense, and it is this gap that allows for many of the above constru Roughly speaking, -closeness constrains expressivity on the order of the number of neurons or the dimension model, and this notion constrains expressivity on the order of the number of parameters over the logarithm of number of properties. This is the information theoretic limit up to a multiplicative constant, as the number of b model is proportional to its parameter count and a table storing properties for entities requires the logarithm number of properties bits per property per entity.

Because these constructions are information theoretically tight, they are the kind of thing transformers might a learn. As the authors note, dictionary learning approaches to untangling such solutions on the residual stream benefit from using a thresholding activation function (rounding interference to 0), which is a stronger constrai a ReLU activation function (which allows for significant negative interference).

IMPROVEMENTS TO ATTRIBUTION PATCHING

Activation patching (see e.g. Zhang & Nanda, 2023) is a common method for finding the components of an LL contribute to a behavior of interest. Briefly, one runs a forward pass of the LLM on some prompt with an expec answer (e.g. "The Eiffel Tower is in the city of" should produce "Paris"), modifies the activation of some compor and then reruns the remainder of that forward pass. Typically, the modified activation is copied from a forwarc on a different prompt. If the new forward pass no longer gives the expected answer, then the component is pa responsible for the original behavior. Although this method is conceptually simple and persuasive, it is expens we want to find one out of 1,000 neurons contributing to some behavior, we would need to run 1,000 forward passes.

Attribution patching (see also Syed et al., 2023) is an efficient approximation to activation patching using grad can be thought of as activation patching where the remainder of the forward pass is replaced with a linear approximation from a backwards pass. With attribution patching, we can examine the approximate influence c component after just one forward and one backward pass. Unfortunately, the approximation isn't always accur

In "AtP*: An efficient and scalable method for localizing LLM behaviour to components", Kramár, Lieberum, Sh Nanda investigate failure modes of attribution patching, and develop two fixes that improve its accuracy withc losing too much efficiency.

- The first failure mode involves nonlinearities immediately following the modified activation, whose gra almost vanish near the original prompt. The paper focuses on the softmax in attention, and demonstra that running the forward pass with the modified activations past the softmax and then taking the grad from afterwards improves the approximation considerably.
    - Since this involves running part of a forward pass through attention for each modified activatic much more expensive than naive attribution patching, but still much cheaper than a full forwar for each modification.
    - This general failure mode also showed up in some of our internal experiments using attributior patching in 1L models, where we found that LayerNorm saturation also caused approximation failures.
- The second failure mode involves cancellation between direct and indirect effects. The paper works arc this by running one backwards pass per layer, skipping that layer when computing gradients, and then averaging the effects across the passes.

They validate their improvements by comparing their approximations to nodes' influences with the "ground tr activation patching, on Pythia-12B. They also demonstrate that attribution patching is more efficient than som clever ways of approximating activation patching.

This paper – and the attribution patching strategy in general – is relevant to anybody who wants to do circuit f at scale.

SELF-REPAIR AND LAYERNORM

Besides computation costs, measuring the effect of a component through ablation is tricky; a later component "self-repair" the impact of an ablation and leave the model output mostly unchanged. In Explorations of Self-F in Language Models, Rushing and Nanda quantify self-repair by measuring the difference between ablating ar attention head and the direct effect of what the head writes out to the residual stream on the logit of the next in pretraining data.

Surprisingly, they find that the non-linearity in LayerNorm explains ~30% of the self-repair that occurs in the 2 tokens with the highest direct effect. Multiple components of the model push to predict the same token so ab just one of them reduces the norm of the residual stream while largely preserving its direction. This lowers the LayerNorm scaling factor, increasing the logits from the other components and counteracting some of the imp ablation.

McGrath et al speculated that self-repair might make the model more robust during training while McDougall al identified "copy suppression heads" that reduced model overconfidence in a way that shows up as self-repa when ablating earlier copying heads. These results suggest that at least some of the self-repair phenomenon i byproduct of LayerNorm, which may be useful for other reasons, not a behavior which received direct optimiz pressure.